FACULTY OF ENGINEERING AND BASIC SCIENCES
ACADEMIC PROGRAM: DATA ENGINEERING AND ARTIFICIAL INTELLIGENCE

COURSE: ETL (G01)
U2 Activity-3: Building a Reliable Data Pipeline with Great Expectations and Apache Airflow - From Data Quality Rules to Orchestrated Pipeline Execution

1. Purpose

Part 1 introduced the orchestration model: tasks, dependencies, DAGs, DAG Runs, Task Instances, execution states, logs, and controlled failure. Part 2 now integrates those orchestration concepts with the data-quality and ETL work already developed in Unit 2.
The practice uses one analytical requirement as a complete end-to-end thread. This deliberately reduces the analytical scope so that the class can focus on the engineering behavior of a reliable pipeline rather than on building many unrelated transformations.

Analytical Requirement R2 Analyze hiring outcomes according to technology.

The pipeline will identify the data required by R2, profile those attributes, formalize quality risks, apply raw-data validation, transform the data, validate the prepared result, persist trusted data, produce an analytical output, and finally orchestrate the complete process with Apache Airflow.

2. Learning Outcomes

By the end of this guided practice, students should be able to:

- Trace one analytical requirement from required attributes to an executable batch pipeline.
- Use profiling evidence to justify data-quality risks and validation rules.
- Implement a raw-data validation gate with Great Expectations.
- Apply documented transformation rules without using cleaning to hide invalid source data.
- Implement a prepared-data validation gate before loading.
- Represent the complete process as Airflow tasks and explicit dependencies.
- Use validation severity to determine whether the workflow continues or stops.
- Inspect successful and failed DAG Runs using task states and logs.
- Demonstrate a safe rerun strategy for the file-based target used in this practice.
- Produce an analytical result that answers R2 using validated prepared data.

3. Practice Architecture

R2 - Analyze hiring outcomes according to technology
|
v
Required attributes
|
v
Targeted profiling
|
v

Quality risks / rules
|
v
EXTRACT
|
v
RAW VALIDATION GATE
|
v
TRANSFORM
|
v
PREPARED VALIDATION GATE
|
v
LOAD
|
v
R2 ANALYTICAL OUTPUT

Orchestrated by Airflow

Key Idea Airflow is introduced after the engineering process has been defined. The DAG does not invent the pipeline logic; it operationalizes a pipeline that has already been reasoned about.

4. Prerequisites

- Part 1 Airflow environment is working with Apache Airflow 3.1.8.
- The local project has a shared data folder mounted at /opt/airflow/data inside the Airflow containers.
- Python code is edited in VS Code and executed by Airflow inside Docker.
- The candidate dataset is available as data/candidates.csv and uses semicolon (;) as the source delimiter.
- Great Expectations Core 1.23.x and pandas are available in the Airflow image.

Engineering Recommendation Pin package versions in the Docker image or requirements file. Reproducibility begins with a reproducible execution environment.

5. Environment Extension for Part 2

If the Airflow image used in Part 1 does not yet contain pandas and Great Expectations, extend the image before running this practice. A custom image is preferable to installing packages every time a container starts.
Example requirements.txt:

```
pandas
great_expectations==1.23.1
```

Example Dockerfile:

```
FROM apache/airflow:3.1.8

COPY requirements.txt /requirements.txt

RUN pip install --no-cache-dir \
    "apache-airflow==3.1.8" \
    -r /requirements.txt
```

Use the course Docker Compose configuration to build this image, then restart the Airflow services. The exact Compose edit depends on the file used in Part 1; do not replace a working Airflow 3.1.8 configuration with an older Airflow 2.x file.

Checkpoint Before continuing, verify inside the Airflow container that Python can import pandas and great_expectations without error.

PART A - FROM R2 TO AN EXECUTABLE DATA CONTRACT

Step 1 - Start from the Analytical Requirement

R2 asks the system to analyze hiring outcomes according to technology. Before writing code, identify the minimum data required to answer that question.

Purpose	Required Attribute
Group applications by technology	Technology
Evaluate the first score condition in the hiring rule	Code Challenge Score
Evaluate the second score condition in the hiring rule	Technical Interview Score
Represent the hiring outcome	HIRED (derived attribute)

The documented hiring rule is:

```
HIRED = 1
if Code Challenge Score >= 7
and Technical Interview Score >= 7

otherwise
```

```
HIRED = 0
```

Engineering Question Which of these attributes come directly from the source, and which one must be derived by transformation?

Step 2 - Perform Targeted Profiling

The goal is not to repeat a full exploratory analysis of every source column. Profile the attributes required by R2 because those attributes directly affect the analytical result.

```
import pandas as pd

df = pd.read_csv(
    "data/candidates.csv",
    sep=";",
)

required_columns = [
    "Technology",
    "Code Challenge Score",
    "Technical Interview Score",
]

display(df[required_columns].head())
df[required_columns].info()

display(
    df[[
        "Code Challenge Score",
        "Technical Interview Score",
    ]].describe()
)

display(
    df[required_columns]
    .isna()
    .mean()
    .sort_values(ascending=False)
)

display(
    df["Technology"]
```

```
    .value_counts(dropna=False)
)
```

Key Idea Profiling describes what the data look like. Validation evaluates the data against a declared rule and threshold.

Step 3 - Connect Profiling Evidence to Quality Risk

Required Attribute	Evidence to Inspect	Quality Risk for R2
Technology	Missing values, blank values, category representation	Applications cannot be grouped reliably by technology.
Code Challenge Score	Missingness and range	HIRED can be calculated incorrectly.
Technical Interview Score	Missingness and range	HIRED can be calculated incorrectly.

The quality risk is not simply a technical observation. It explains how a data defect could prevent or distort the analytical requirement.

Step 4 - Formalize the Raw Quality Rules

ID	Attribute	Quality Rule	Dimension	Threshold	Severity
DQ01	Technology	Technology must be non-null in at least 98% of rows.	Completeness	>= 98%	Warning
DQ02	Code Challenge Score	Values must be between 0 and 10 inclusive.	Validity	100%	Critical
DQ03	Technical Interview Score	Values must be between 0 and 10 inclusive.	Validity	100%	Critical

Engineering Practice A threshold is meaningful only when its operational consequence is defined. In this practice, Critical failures stop the pipeline. Warning failures are logged and may continue according to the documented policy.

Step 5 - Define the Raw Validation Policy

Situation	Pipeline Response
All raw Expectations pass	Continue to transformation.
Only warning-level Expectations fail	Log the warning and continue according to policy.
At least one critical Expectation fails	Fail the Airflow validation task and block downstream transformation.

Important A retry is not a correction mechanism for deterministic bad data. If the same invalid score is read again, retrying the same validation will normally fail again.

PART B – PROCESSING LOGIC BEFORE ORCHESTRATION

Step 6 - Prepare the Working Directories

The practice uses persisted files between tasks:

```
airflow-docker-2026-2/
│
├── dags/
│   └── reliable_candidate_pipeline.py
│
├── src/
│   ├── 01_extract_candidates.py
│   ├── 02_validate_raw_candidates.py
│   ├── 03_transform_candidates.py
│   ├── 04_validate_prepared_candidates.py
│   ├── 05_load_candidates.py
│   └── 06_build_r2_summary.py
│
├── data/
│   ├── candidates.csv
│   ├── work/
│   └── output/
│
├── Dockerfile
├── requirements.txt
├── .env
└── docker-compose.yaml
```

Engineering Recommendation Persist intermediate datasets in shared storage and pass lightweight references (such as file paths) between tasks. Do not use the orchestrator as a transport mechanism for large DataFrames.

Step 7 - Extraction Responsibility

The extraction stage reads the source, verifies that the attributes required by R2 exist, selects only those attributes, and writes a raw working file.

Expected output:

```
/opt/airflow/data/work/candidates_r2_raw.csv
```

A missing required column is treated as a blocking source-contract failure because the pipeline cannot answer R2 without the required data.

Step 8 - Raw Validation with Great Expectations

The raw validation task evaluates DQ01-DQ03 before any cleaning or business transformation. The validation result is interpreted using Expectation severity.

```
Raw data
  |
  v
GX Expectation Suite
  |
  v
Validation Result
  |
  +-- maximum failed severity = Critical --> FAIL task
  |
  +-- maximum failed severity = Warning  --> LOG + continue
  |
  `-- no failures                        --> continue
```

Key Idea GX detects and reports the data-quality condition. The pipeline policy decides whether that condition is blocking. Airflow enforces the resulting workflow behavior.

Step 9 - Transformation for R2

The transformation stage applies only documented rules required to make the dataset analytically usable for R2.

1. Trim leading and trailing whitespace from Technology.

2. Convert blank Technology strings to missing values.
3. Remove rows with missing Technology because such rows cannot be assigned to a technology group for R2.
4. Derive HIRED using the documented hiring rule.

Important Do not change invalid score values merely to make validation pass. A score outside the documented range is a source-quality problem, not a formatting problem.

Step 10 - Prepared Validation

The prepared-data validation gate answers a different question: Did transformation produce data suitable for loading and for answering R2?

Prepared Rule	Reason
Technology must not be null.	R2 groups results by Technology.
HIRED must contain only 0 or 1.	HIRED is a binary analytical attribute.
HIRED must not be null.	Every retained application must have a defined hiring outcome.

Conceptual Distinction Raw validation protects transformation from unacceptable input. Prepared validation protects the target from unacceptable transformed data.

Step 11 - Load Strategy

For this guided practice, the validated prepared dataset is loaded to a file-based target. The Workshop-2 will extend the same design toward a dimensional Data Warehouse.

```
/opt/airflow/data/output/candidates_r2_validated.csv
```

The file is overwritten on each successful load rather than appended. This makes the guided example safe to rerun without silently duplicating records.

Engineering Recommendation In database targets, safe reruns may require a batch key, controlled replace strategy, upsert, or another idempotent loading design. The correct strategy depends on the target and business semantics.

Step 12 - Produce the Analytical Output

After loading, build a compact result that answers R2. The output should be created from the validated loaded dataset, not from the original source.

Metric	Definition
Applications	Number of retained applications for each Technology.
Hired	Sum of HIRED for each Technology.
Hiring Rate	Mean of HIRED for each Technology, expressed as a percentage.

Checkpoint Before Airflow is considered, verify that the process can logically produce trusted data that answer R2. Orchestration cannot repair a poorly defined pipeline.

PART C - IMPLEMENT THE PIPELINE IN APACHE AIRFLOW

Step 13 - Map Engineering Stages to Airflow Tasks

Airflow Task	Engineering Responsibility
extract_candidates	Read the source and create the raw working dataset.
validate_raw_candidates	Apply the raw GX quality gate.
transform_candidates	Normalize Technology and derive HIRED.
validate_prepared_candidates	Verify the transformed dataset before loading.
load_candidates	Persist validated prepared data.
build_r2_summary	Produce the analytical output for R2 from loaded data.

Step 14 - Create the DAG File

Create:

```
dags/reliable_candidate_pipeline.py
```

The complete script is included in Appendix A and is also provided as a separate .py file with this activity.

Step 15 - Airflow 3 TaskFlow Imports

```
import pendulum

from airflow.sdk import dag, task
```

The practice uses the Airflow 3 public Task SDK. Decorated task functions become Airflow tasks when the DAG is constructed.

Engineering Recommendation Keep the DAG readable. A DAG should make workflow structure and operational policy easy to inspect. As the project grows, move reusable extraction, transformation, validation, and loading logic into src/ modules.

Step 16 - Define the DAG

```
@dag(
    dag_id="reliable_candidate_pipeline_r2",
    schedule=None,
    start_date=pendulum.datetime(
        2026, 1, 1, tz="UTC"
    ),
    catchup=False,
    tags=["etl", "gx", "reliable-pipeline", "r2"],
)
def reliable_candidate_pipeline_r2():
    ...
```

schedule=None is intentional: this guided practice uses manual DAG Runs so that each execution can be inspected. Scheduling is an operational concern that can be added after the workflow behavior is understood.

Step 17 - Build Dependencies Around Validation Gates

```
extract_candidates
        |
        v
validate_raw_candidates
        |
        v
transform_candidates
        |
        v
validate_prepared_candidates
        |
        v
load_candidates
        |
        v
build_r2_summary
```

The most important dependencies are the validation gates. Transformation must not proceed until the raw validation policy is satisfied, and loading must not proceed until prepared validation succeeds.

Step 18 - First Execution: Successful Run

- Confirm SOURCE_FILENAME points to candidates.csv.
- Save the DAG file in dags/.
- Wait for Airflow to parse the DAG.
- Open the Airflow Web UI and locate reliable_candidate_pipeline_r2.
- Inspect the Graph view before running it.
- Trigger a new DAG Run.
- Observe the task states as the workflow progresses.
- Open the logs for both validation tasks.
- Confirm that the output files are created in data/output/.

Expected conceptual result:

```
extract_candidates              SUCCESS
        |
validate_raw_candidates         SUCCESS
        |
transform_candidates            SUCCESS
        |
validate_prepared_candidates    SUCCESS
        |
load_candidates                 SUCCESS
        |
build_r2_summary                SUCCESS
```

Evidence to Capture Graph/Grid view of the successful DAG Run, validation-task logs, row counts, and the generated R2 summary file.

Step 19 - Inspect the Analytical Result

Open r2_hiring_by_technology.csv and verify that it contains the metrics required by R2. At this point, the result is supported by a chain of data-quality and execution evidence.

Engineering Question Why is this result more trustworthy than running a groupby directly on the original source CSV?

Step 20 - Controlled Quality Failure

Create a separate bad-data file. Do not modify the original candidates.csv.

```
import pandas as pd

df_bad = pd.read_csv(
    "data/candidates.csv",
    sep=";",
)

df_bad.loc[
    df_bad.index[0],
    "Code Challenge Score",
] = 15

df_bad.to_csv(
    "data/candidates_bad.csv",
    sep=";",
    index=False,
)
```

Then change the DAG configuration from candidates.csv to candidates_bad.csv, save the DAG, wait for Airflow to parse the new version, and trigger a NEW DAG Run.

```
Expected conceptual result:

extract_candidates              SUCCESS
        |
validate_raw_candidates         FAILED
        |
transform_candidates            does not proceed normally
        |
validate_prepared_candidates    does not proceed normally
        |
load_candidates                 does not proceed normally
        |
build_r2_summary                does not proceed normally
```

Key Observation The data problem is detected by Great Expectations. The pipeline policy classifies the critical failure as blocking. Airflow then enforces the dependency behavior.

Step 21 - Investigate the Failed Run

- Open the failed DAG Run.
- Identify the first failed task.
- Inspect validate_raw_candidates logs.
- Locate the GX validation statistics and maximum failure severity.

- Confirm that Code Challenge Score violated DQ02.
- Observe the actual Airflow states assigned to downstream tasks.
- Explain why the pipeline did not proceed to Load.

Observability Questions What ran? When? Where did the workflow first fail? Why? What happened to the downstream tasks?

Step 22 - Restore the Successful Source

Change SOURCE_FILENAME back to candidates.csv, save the DAG, and trigger another run. Confirm that the workflow returns to a successful state.

```
Run 1 -> Successful workflow
Run 2 -> Controlled critical quality failure
Run 3 -> Successful workflow after restoration
```

Each DAG Run preserves separate execution evidence even though the workflow definition represents the same logical pipeline.

Step 23 - Retry Policy Discussion

This guided pipeline uses local files and deterministic validations, so retrying failed quality gates would normally be inappropriate.

Failure	Retry?	Reason
Code Challenge Score = 15	No	The same input will fail the same deterministic rule.
Missing required column	No	The pipeline contract cannot be satisfied by retrying.
Temporary remote database outage	Yes, usually	The external condition may recover without changing data or code.
Temporary network interruption	Yes, usually	The failure may be transient.

Engineering Principle Configure retries for failures that can reasonably become successful on a later attempt. Do not add retries mechanically to every task.

Step 24 - Safe Rerun / Repeatability Test

- Run the successful pipeline twice using the same candidates.csv input.

- Compare the row count of candidates_r2_validated.csv after both runs.
- Confirm that the file-based load overwrote the previous target instead of appending duplicate rows.
- Confirm that r2_hiring_by_technology.csv is reproducible under the same input and transformation rules.

Key Idea Repeatability means predictable execution under the same conditions. Idempotent or controlled loading prevents retries and reruns from creating unintended side effects.

Step 25 - Engineering Reflection

- Which parts of the pipeline were determined directly by R2?
- Why is Technology completeness a warning while invalid score ranges are critical in this practice?
- What is the difference between raw validation and prepared validation?
- Why should an invalid score not be corrected simply to make GX pass?
- Which component detected the controlled data-quality problem?
- Which component enforced the downstream execution behavior?
- Why does the pipeline exchange file paths rather than complete DataFrames between tasks?
- Why are retries not appropriate for the controlled DQ02 failure?
- How does the load strategy make this guided pipeline safe to rerun?
- What would need to change when the file-based target is replaced by a Data Warehouse?

Appendix A - Complete Airflow DAG Script

The following script is the complete guided implementation. Comments intentionally mark each engineering stage so that the relationship between pipeline design and Airflow orchestration remains visible.

```
"""
Reliable Candidate Pipeline - Guided Programming Practice Part 2

Course: ETL - Data Engineering and Artificial Intelligence
Airflow: 3.1.8
Great Expectations: 1.23.x

Analytical requirement:
R2 - Analyze hiring outcomes according to technology.

The DAG exchanges FILE PATHS between tasks, not complete DataFrames.
All tasks read/write files under /opt/airflow/data, which must be mounted
as a shared Docker volume.
"""

from datetime import timedelta
from pathlib import Path
import shutil

import great_expectations as gx
import pandas as pd
import pendulum

from airflow.sdk import dag, task
from great_expectations.expectations.metadata_types import FailureSeverity


#
============================================================
==
# GLOBAL CONFIGURATION
#
============================================================
==

DATA_DIR = Path("/opt/airflow/data")
WORK_DIR = DATA_DIR / "work"
OUTPUT_DIR = DATA_DIR / "output"

# Successful run:
```

```
SOURCE_FILENAME = "candidates.csv"

# Controlled-failure run:
# After creating candidates_bad.csv, change the previous line to:
# SOURCE_FILENAME = "candidates_bad.csv"

SOURCE_PATH = DATA_DIR / SOURCE_FILENAME

RAW_PATH = WORK_DIR / "candidates_r2_raw.csv"
PREPARED_PATH = WORK_DIR / "candidates_r2_prepared.csv"
TARGET_PATH = OUTPUT_DIR / "candidates_r2_validated.csv"
SUMMARY_PATH = OUTPUT_DIR / "r2_hiring_by_technology.csv"

REQUIRED_COLUMNS = [
    "Technology",
    "Code Challenge Score",
    "Technical Interview Score",
]


#
============================================================
==
# HELPER: BUILD AND RUN A GX VALIDATION
#
============================================================
==

def run_gx_validation(dataframe, stage):
    """
    Build a small ephemeral GX validation for one pipeline stage.

    stage = "raw"
        Validates source attributes required by R2.

    stage = "prepared"
        Validates the transformed analytical dataset.

    The GX context is ephemeral because this guided practice focuses on
    orchestration. In a larger project, suites and validation definitions
    should normally be persisted and maintained as project assets.
    """

    context = gx.get_context(mode="ephemeral")
```

```
data_source = context.data_sources.add_pandas(
    name=f"{stage}_candidate_source"
)

data_asset = data_source.add_dataframe_asset(
    name=f"{stage}_candidate_asset"
)

batch_definition = data_asset.add_batch_definition_whole_dataframe(
    f"{stage}_candidate_batch"
)

suite = gx.ExpectationSuite(
    name=f"{stage}_candidate_suite"
)
suite = context.suites.add(suite)

if stage == "raw":
    # DQ01 - Technology completeness.
    # Warning: a limited amount of missing Technology is tolerated.
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToNotBeNull(
            column="Technology",
            mostly=0.98,
            severity="warning",
        )
    )

    # DQ02 - Code Challenge Score validity.
    # Critical because HIRED depends directly on this attribute.
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToBeBetween(
            column="Code Challenge Score",
            min_value=0,
            max_value=10,
            severity="critical",
        )
    )

    # DQ03 - Technical Interview Score validity.
    # Critical because HIRED depends directly on this attribute.
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToBeBetween(
            column="Technical Interview Score",
            min_value=0,
```

```
            max_value=10,
            severity="critical",
        )
    )

elif stage == "prepared":
    # Prepared data must contain Technology because R2 groups by it.
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToNotBeNull(
            column="Technology",
            severity="critical",
        )
    )

    # HIRED is a derived binary attribute.
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToBeInSet(
            column="HIRED",
            value_set=[0, 1],
            severity="critical",
        )
    )

    # HIRED must be complete before the data are loaded.
    suite.add_expectation(
        gx.expectations.ExpectColumnValuesToNotBeNull(
            column="HIRED",
            severity="critical",
        )
    )

else:
    raise ValueError(f"Unknown validation stage: {stage}")

validation_definition = gx.ValidationDefinition(
    name=f"{stage}_candidate_validation",
    data=batch_definition,
    suite=suite,
)

validation_definition = context.validation_definitions.add(
    validation_definition
)

result = validation_definition.run(
```

```
        batch_parameters={"dataframe": dataframe},
        result_format={"result_format": "SUMMARY"},
    )

    print(f"GX stage: {stage}")
    print(f"Validation success: {result.success}")
    print(f"Statistics: {result.statistics}")

    max_failure = result.get_max_severity_failure()
    print(f"Maximum failure severity: {max_failure}")

    return result, max_failure


#
============================================================
==
# DAG DEFINITION
#
============================================================
==

@dag(
    dag_id="reliable_candidate_pipeline_r2",
    schedule=None,
    start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
    catchup=False,
    tags=["etl", "gx", "reliable-pipeline", "r2"],
)
def reliable_candidate_pipeline_r2():

    #
============================================================
    # STAGE 1 - EXTRACT
    #
============================================================
    @task
    def extract_candidates():
        """
        Read the source CSV, verify the attributes required by R2,
        and persist a raw working file.

        The task returns only a FILE PATH. It does not send the full
        DataFrame through Airflow.
        """
```

```
    WORK_DIR.mkdir(parents=True, exist_ok=True)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    if not SOURCE_PATH.exists():
        raise FileNotFoundError(
            f"Source file not found: {SOURCE_PATH}"
        )

    df = pd.read_csv(SOURCE_PATH, sep=";")

    missing_columns = [
        column
        for column in REQUIRED_COLUMNS
        if column not in df.columns
    ]

    if missing_columns:
        raise ValueError(
            f"Required columns are missing: {missing_columns}"
        )

    raw_df = df[REQUIRED_COLUMNS].copy()
    raw_df.to_csv(RAW_PATH, index=False)

    print(f"Source: {SOURCE_PATH}")
    print(f"Rows extracted: {len(raw_df)}")
    print(f"Raw working file: {RAW_PATH}")

    return str(RAW_PATH)

#
# STAGE 2 - RAW VALIDATION GATE
#
@task
def validate_raw_candidates(raw_path):
    """
    Validate the incoming data BEFORE transformation.

    Critical GX failures stop the pipeline.
    Warning-level failures are logged but do not block execution.
    """
```

```
    df = pd.read_csv(raw_path)

    result, max_failure = run_gx_validation(
        dataframe=df,
        stage="raw",
    )

    if max_failure == FailureSeverity.CRITICAL:
        raise ValueError(
            "Raw validation failed with CRITICAL severity. "
            "Transformation is blocked."
        )

    if max_failure == FailureSeverity.WARNING:
        print(
            "Raw validation contains WARNING-level failures. "
            "The documented policy allows the pipeline to continue."
        )

    print("Raw validation gate passed according to policy.")

# ============================================================
# STAGE 3 - TRANSFORM
# ============================================================
@task
def transform_candidates(raw_path):
    """
    Apply documented transformation rules for R2.

    1. Normalize Technology text.
    2. Remove rows without Technology because they cannot support
       grouping by technology.
    3. Derive HIRED using the documented business rule:
         Code Challenge Score >= 7
         AND
         Technical Interview Score >= 7
    """

    df = pd.read_csv(raw_path)

    # Normalize Technology.
    df["Technology"] = (
        df["Technology"]
```

```
        .astype("string")
        .str.strip()
    )

    df["Technology"] = df["Technology"].replace("", pd.NA)

    missing_before = int(df["Technology"].isna().sum())

    # R2 cannot group a record by technology if Technology is missing.
    df = df.dropna(subset=["Technology"]).copy()

    # Derive the analytical attribute HIRED.
    df["HIRED"] = (
        (df["Code Challenge Score"] >= 7)
        & (df["Technical Interview Score"] >= 7)
    ).astype(int)

    df.to_csv(PREPARED_PATH, index=False)

    print(f"Rows with missing Technology removed: {missing_before}")
    print(f"Prepared rows: {len(df)}")
    print(f"Prepared working file: {PREPARED_PATH}")

    return str(PREPARED_PATH)

#
# STAGE 4 - PREPARED VALIDATION GATE
#
@task
def validate_prepared_candidates(prepared_path):
    """
    Validate the result of transformation BEFORE loading.

    At this boundary the pipeline verifies that the dataset is
    analytically usable for R2.
    """

    df = pd.read_csv(prepared_path)

    result, max_failure = run_gx_validation(
        dataframe=df,
        stage="prepared",
    )
```

```
    if max_failure == FailureSeverity.CRITICAL:
        raise ValueError(
            "Prepared-data validation failed with CRITICAL severity. "
            "Loading is blocked."
        )

    print("Prepared validation gate passed.")

#
# STAGE 5 - LOAD
#
@task
def load_candidates(prepared_path):
    """
    Persist the validated prepared dataset.

    shutil.copyfile overwrites the target, which makes this simple
    file-based guided example safe to rerun without appending duplicate
    records.
    """

    source = Path(prepared_path)
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

    shutil.copyfile(source, TARGET_PATH)

    loaded_df = pd.read_csv(TARGET_PATH)

    print(f"Validated dataset loaded to: {TARGET_PATH}")
    print(f"Rows loaded: {len(loaded_df)}")

    return str(TARGET_PATH)

#
# STAGE 6 - ANALYTICAL OUTPUT FOR R2
#
@task
def build_r2_summary(target_path):
    """
    Answer R2 using the validated loaded data.
```

```
    Metrics:
    - Applications
    - Hired candidates
    - Hiring rate
    """

    df = pd.read_csv(target_path)

    summary = (
        df.groupby("Technology", as_index=False)
        .agg(
            Applications=("HIRED", "size"),
            Hired=("HIRED", "sum"),
            Hiring_Rate=("HIRED", "mean"),
        )
        .sort_values(
            by="Hiring_Rate",
            ascending=False,
        )
    )

    summary["Hiring_Rate"] = (
        summary["Hiring_Rate"] * 100
    ).round(2)

    summary.to_csv(SUMMARY_PATH, index=False)

    print("R2 - Hiring outcomes according to technology")
    print(summary.head(10).to_string(index=False))
    print(f"Analytical output: {SUMMARY_PATH}")

#
# BUILD THE WORKFLOW
#

raw_path = extract_candidates()

raw_validation = validate_raw_candidates(raw_path)

prepared_path = transform_candidates(raw_path)

# Validation is a gate: transformation may only proceed after
```

```
    # raw validation satisfies the documented policy.
    raw_validation >> prepared_path

    prepared_validation = validate_prepared_candidates(
        prepared_path
    )

    target_path = load_candidates(prepared_path)

    # The target may only be loaded after prepared validation succeeds.
    prepared_validation >> target_path

    build_r2_summary(target_path)


# Make the DAG discoverable by Airflow.
reliable_candidate_pipeline_r2()
```

Appendix B - Controlled Failure Preparation Script

```
import pandas as pd

df_bad = pd.read_csv(
    "data/candidates.csv",
    sep=";",
)

# Inject one deterministic critical violation.
df_bad.loc[
    df_bad.index[0],
    "Code Challenge Score",
] = 15

df_bad.to_csv(
    "data/candidates_bad.csv",
    sep=";",
    index=False,
)

print("Controlled failure dataset created.")
```

Final Synthesis

```
Business / Analytical Requirement
            |
            v
        Required Data
            |
            v
         Profiling
            |
            v
        Quality Risk
            |
            v
       Quality Rule
            |
            v
      GX Expectation
            |
            v
    Validation Result
            |
            v
     Pipeline Policy
            |
            v
    Airflow Task State
            |
            v
    Workflow Behavior
            |
            v
      Trusted Output
```

Unit 2 Synthesis Reliable data pipelines do not assume that data is correct. They verify it. Reliable systems do not assume that execution will succeed. They are designed for what happens when it does not.